Quality Engineering & Quality Control

Module 13 · 1-Day Training Session · Publicis Sapient

SUSTAIN ENGINEERING TRACK

Before We Begin

What does quality mean in production support?

Think about it

When did a production system last feel "high quality" to you — and what made it feel that way?

Consider this

What is the difference between a bug that annoys a user and one that stops the business?

Reflect

Who is responsible for quality on a sustain team — the tester, the developer, or everyone?

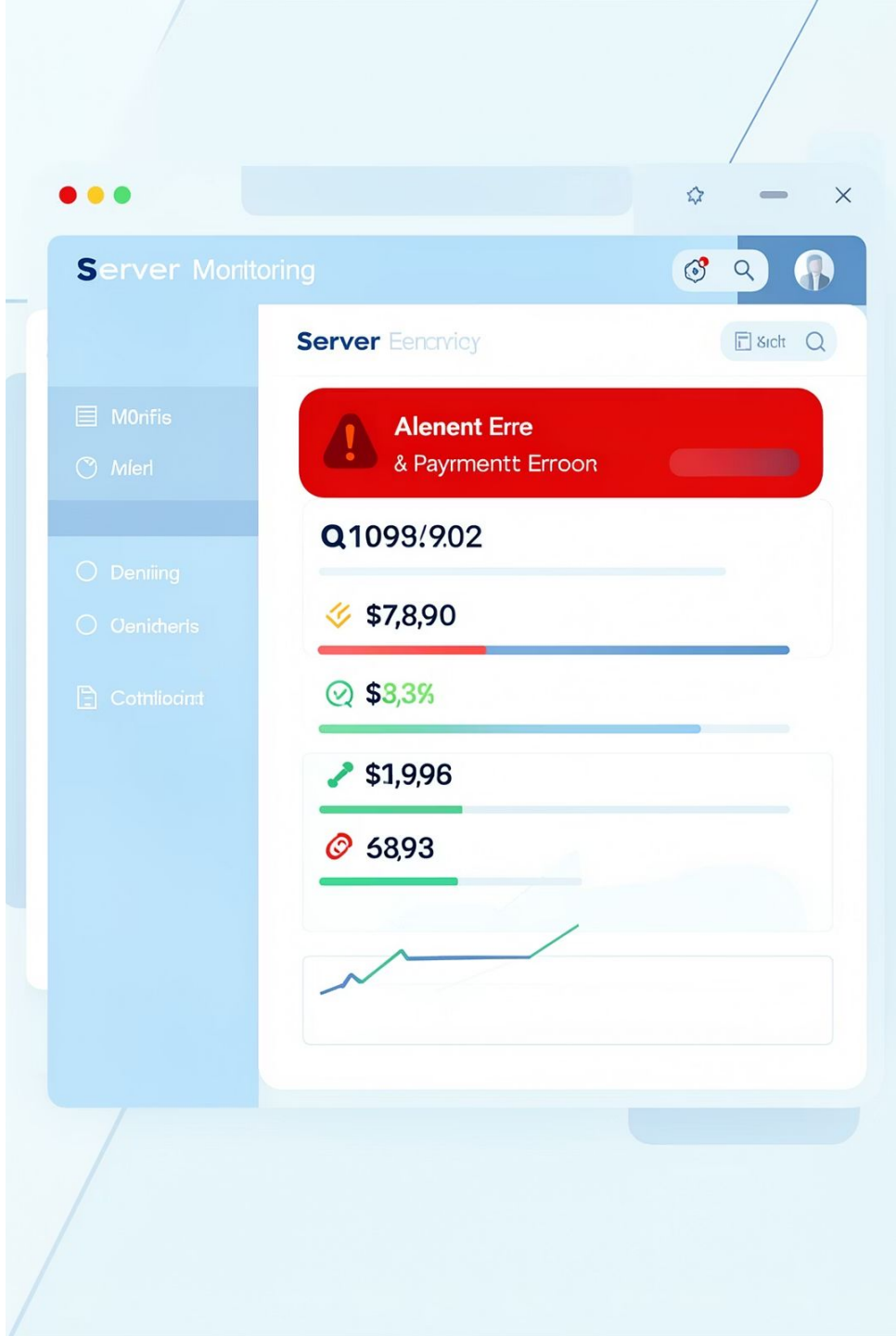
There are no wrong answers. Keep these questions in mind throughout today.

A Deployment Goes Wrong

A routine Tuesday deployment to a payment processing service. No major changes flagged. Monitoring shows green for 4 minutes.

1. **Patch deployed**
   Small config change
2. **Logic error triggered**
   Duplicate charge fires
3. **Customers charged twice**
   Hundreds affected
4. **Incident raised**
   Rollback + refunds begin

⊗ This is a real pattern. It happens when quality is treated as optional.



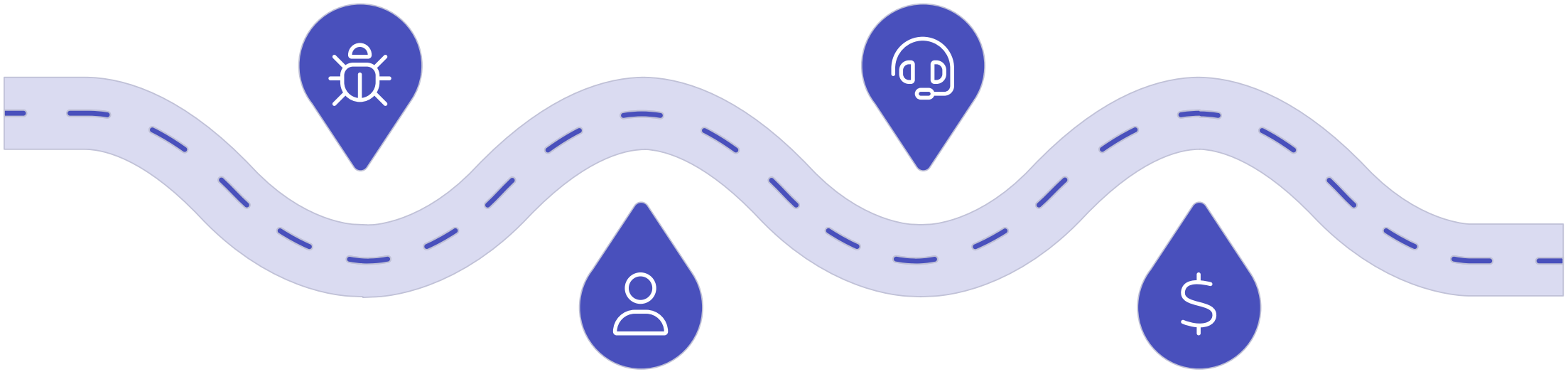
One Small Defect. Big Consequences.

Code Defect

Bug introduced into production

Ops Overload

Support flooded with tickets



User Impact

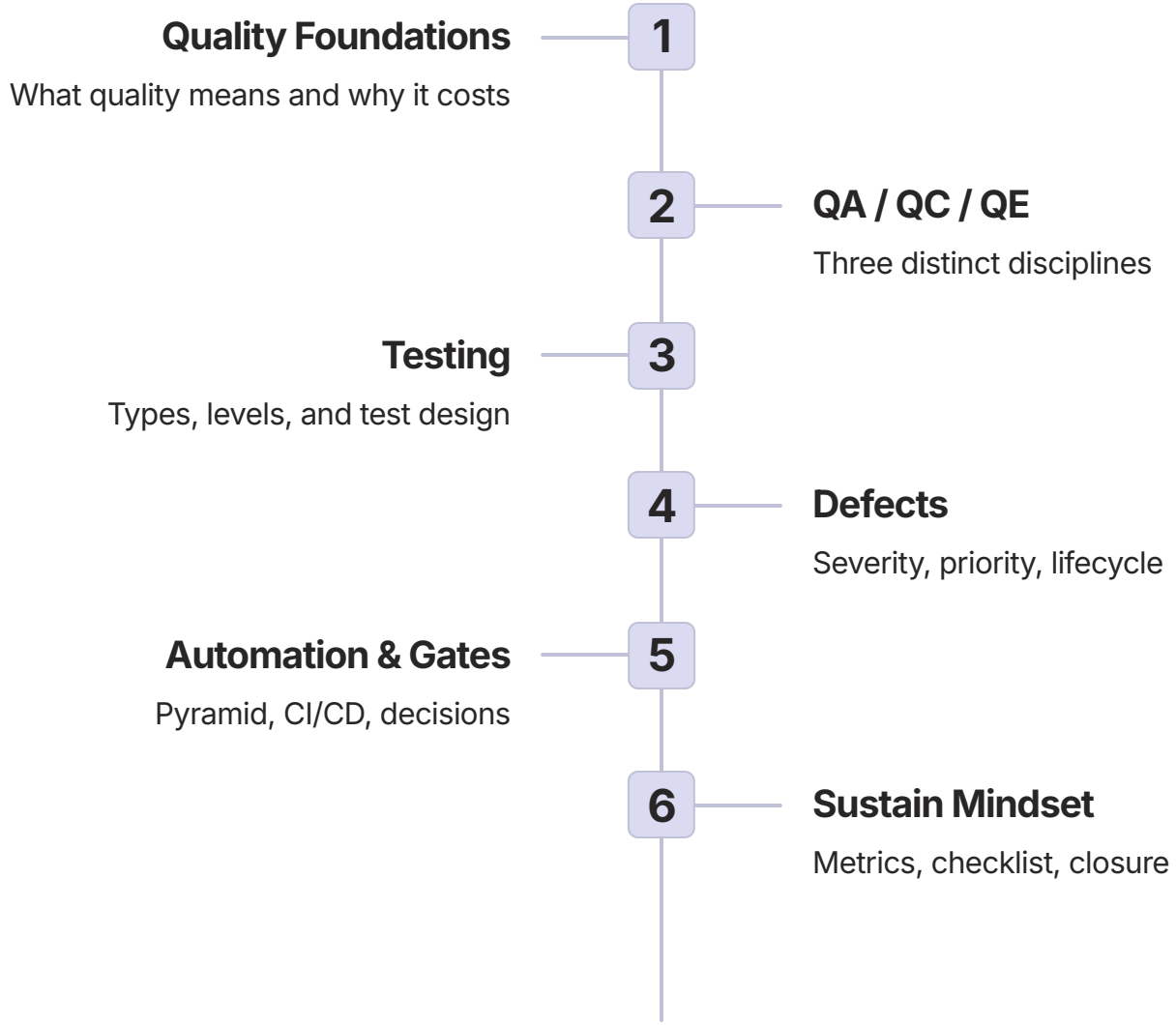
Users see incorrect behavior

Revenue Loss

Refunds rise and revenue drops

Every defect in production travels this chain. The question is how far it gets before someone catches it.

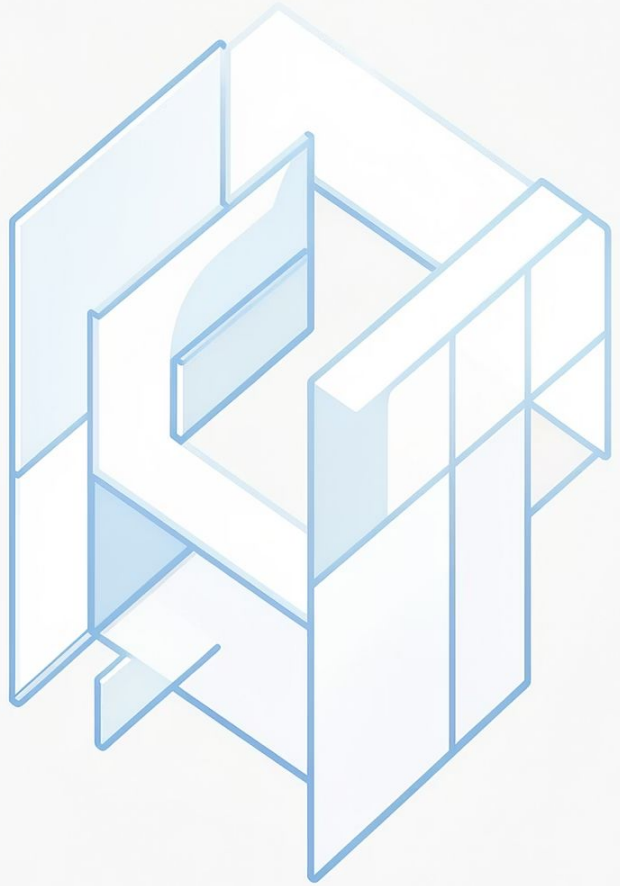
Your Learning Journey Today



SECTION 1 OF 5

Why Quality Matters

Building the foundation: what quality is, what it costs, and what changes in sustain engineering.



What Is Software Quality?

A simple definition

Software quality means the system **does what it should**, **reliably**, under the conditions users actually experience.

Quality is not perfection. It is fitness for purpose.

Meets requirements

Does the right things

Works reliably

Consistent under load

Satisfies users

Feels correct and fast

Quality Has Three Views

User View

Does it work the way I expect?

Fast, intuitive, error-free

Business View

Does it protect revenue and reputation?

Reliable, compliant, available

Engineering View

Is it maintainable and observable?

Testable, modular, monitored

All three views matter. Sustain engineers must balance all of them — every sprint.

Quality Attributes

Correctness

System produces the right output for every valid input

Reliability

Works consistently without unexpected failures

Performance

Responds quickly under expected and peak load

Security

Resists unauthorised access and data leakage

Usability

Intuitive for the intended user without extra effort

Maintainability

Easy to change, test, and understand over time

What Changes in Sustain Engineering?

Existing system, not a blank slate

You inherit decisions made by others, often without full documentation

Production users are already there

Every change risks real people and real data — not a test environment

Complex dependencies

Services, APIs, databases, and third-party systems may break unexpectedly

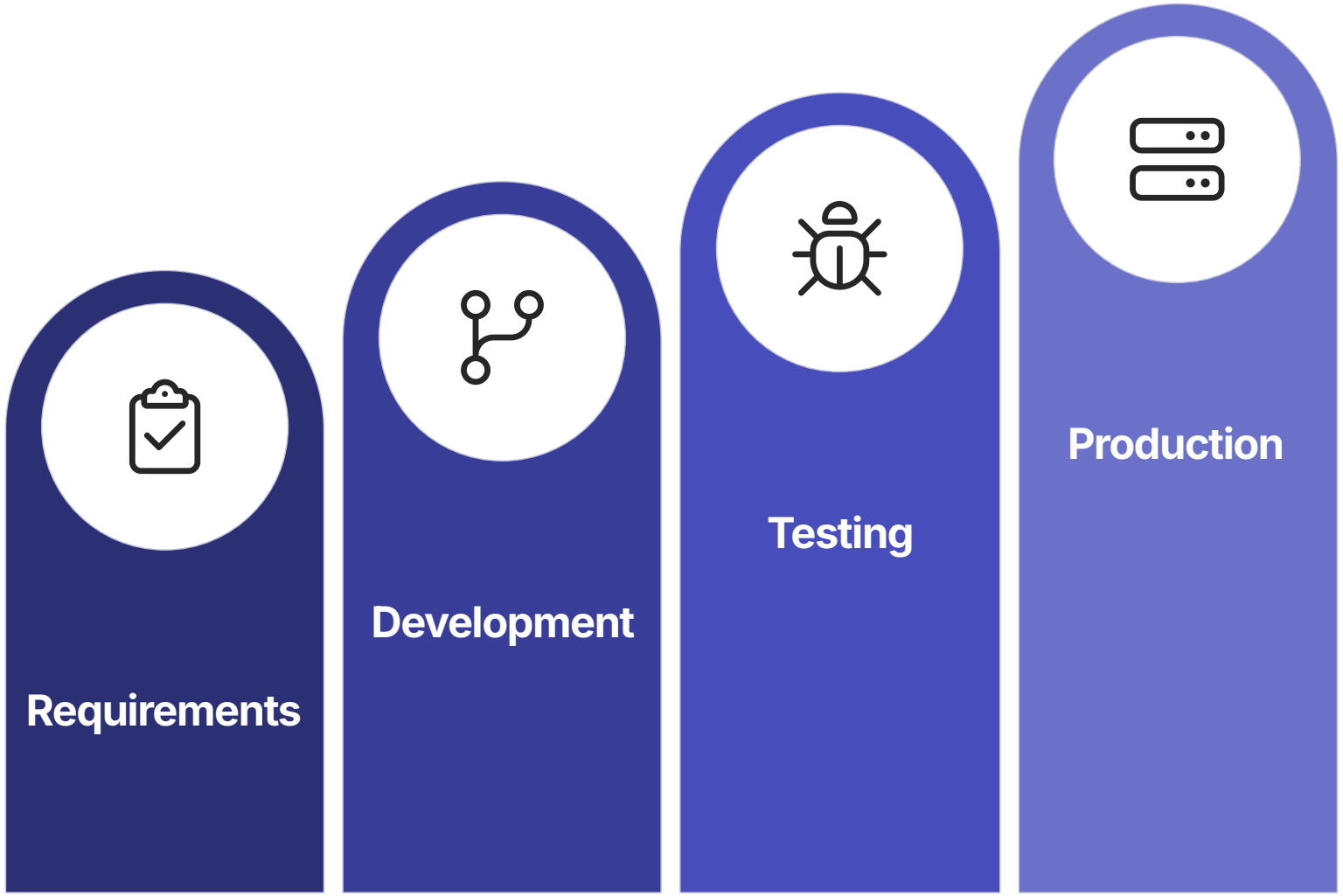
Legacy constraints

Old code, missing tests, tribal knowledge — quality starts from behind

Greenfield vs Sustain Engineering

Dimension	Greenfield	Sustain
Design freedom	High — start fresh	Low — work within constraints
Risk of change	Lower — fewer dependencies	Higher — live users, integrations
System knowledge	Built by the same team	Inherited, often incomplete
Blast radius	Small — limited user base	Large — production at scale
Regression risk	Low early on	High — every change may break something

The Cost of a Defect Over Time



Finding a defect in production costs 10–100x more than finding it during requirements or design. Fix quality early.

Prevention vs Detection

Detection

Finding bugs after they exist

- Code review catches an issue
- Test fails in CI pipeline
- User reports a production error

Prevention

Stopping bugs from being introduced

- Clear acceptance criteria before coding
- Peer design review before implementation
- Automated quality gates before merge

Quality is not only finding bugs. It is building systems where fewer bugs can enter.

Core Quality Principles

Prevention First

Design out defects before they are coded

Early Feedback

Catch issues as close to origin as possible

Risk Focus

Test what matters most, not everything equally

Observability

You cannot improve what you cannot see in production

Continuous Improvement

Every incident is a learning opportunity

The Sustain Quality Mindset

Change Safely

Understand impact before touching production

Prove It

Run the right tests before and after deploy

Learn from It

Improve process so the same issue cannot recur

Observe It

Watch metrics and logs after the change lands

ACTIVITY · 10 MINUTES

Mini Activity: Spot the Quality Risks

⚠ Scenario: Your team is about to push a small config change to a payment microservice in production. It touches the timeout values for an external bank API call.

In pairs, identify 5 possible quality risks in this change. Think about: what could break, who would be affected, and how you would know.

Risk 1

Write it here

Risk 2

Write it here

Risk 3

Write it here

Risk 4

Write it here

Risk 5

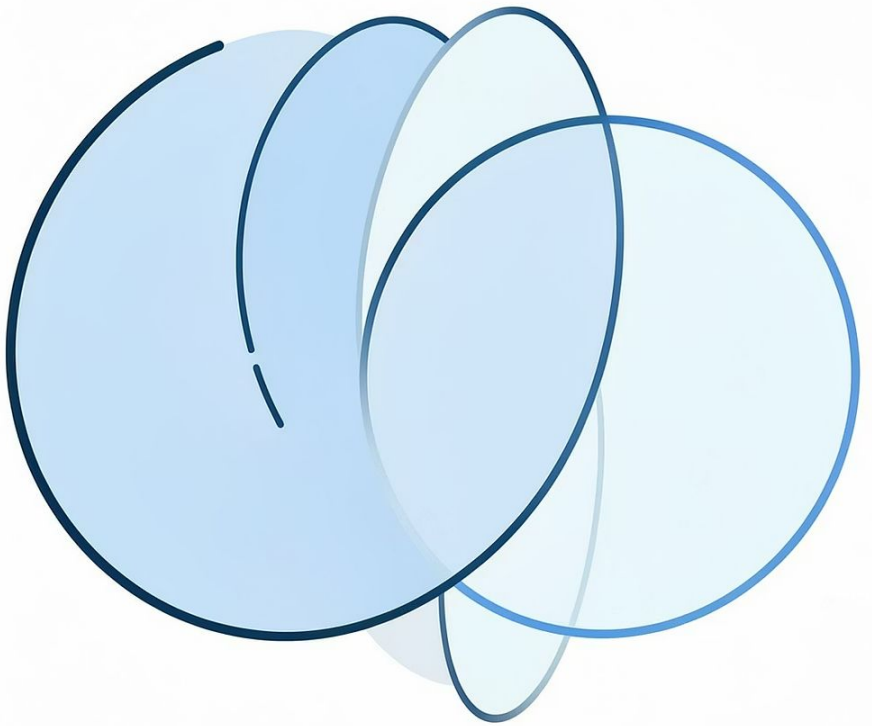
Write it here

Share with the group. Trainer will debrief on the most common blind spots.

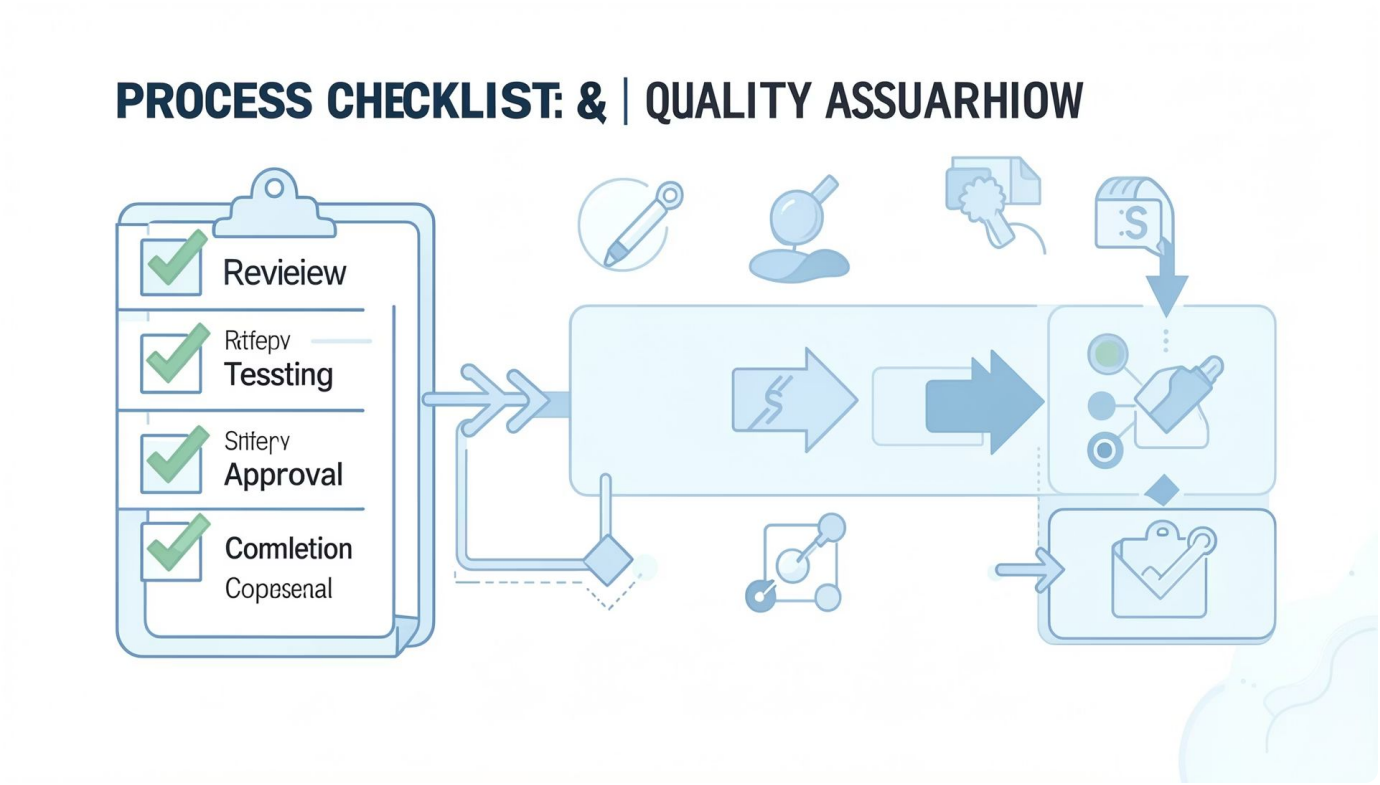
SECTION 2 OF 5

QA vs QC vs QE

Three related but distinct disciplines — and sustain engineers must understand all three.



Quality Assurance (QA)



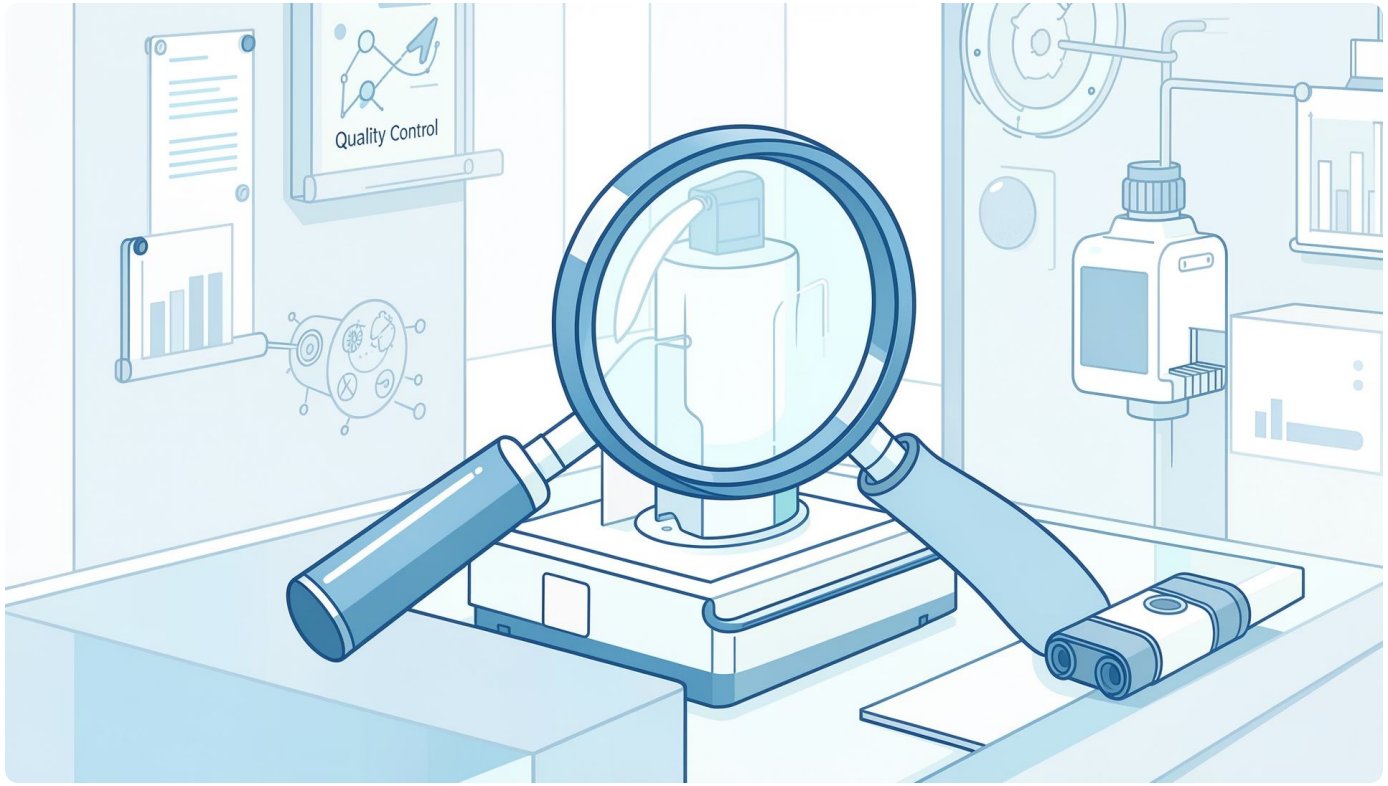
QA is process-focused prevention

QA asks: **"Are we following the right processes to produce quality?"**

- Defines standards and guidelines
- Reviews how work is done, not just what is done
- Catches process failures before they become product failures

QA happens before and during development — not just at the end.

Quality Control (QC)



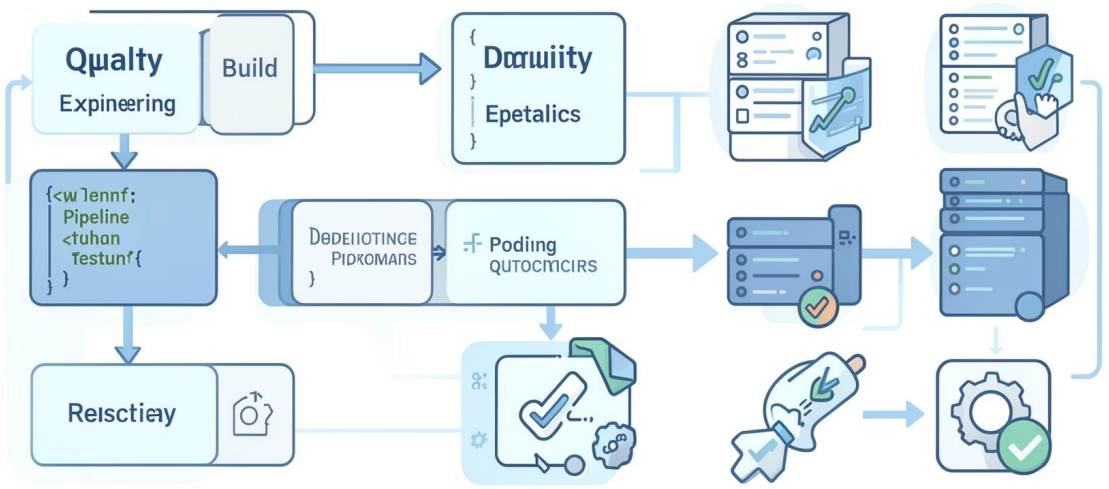
QC is product-focused detection

QC asks: **"Does this specific output meet the required standard?"**

- Inspects the actual build or release
- Runs tests to find defects in the product
- Reactive — detects problems that already exist

QC is what most people picture when they say "testing".

Quality Engineering (QE)



QE engineers quality into the delivery lifecycle

QE asks: **"How do we build systems where quality is automatic, fast, and continuous?"**

- Builds automated tests and quality gates
- Embeds quality into the CI/CD pipeline
- Partners with developers from the start

> QE is proactive — quality is designed in, not bolted on later.

QA vs QC vs QE at a Glance

Dimension	QA	QC	QE
Focus	Process	Product	Engineering pipeline
Timing	Before and during build	After build	Throughout all stages
Examples	Code review process, definition of done	Test execution, bug reporting	Automated tests, quality gates
Owner mindset	Are we doing it right?	Is this output correct?	How do we make quality automatic?

A Simple Analogy

QA — The Recipe

A restaurant defines standard recipes, hygiene procedures, and training programmes so chefs consistently produce safe food.

Software: Coding standards, review checklists, definition of done

QC — The Taste Test

Before a dish leaves the kitchen, a chef tastes it and checks presentation against the standard.

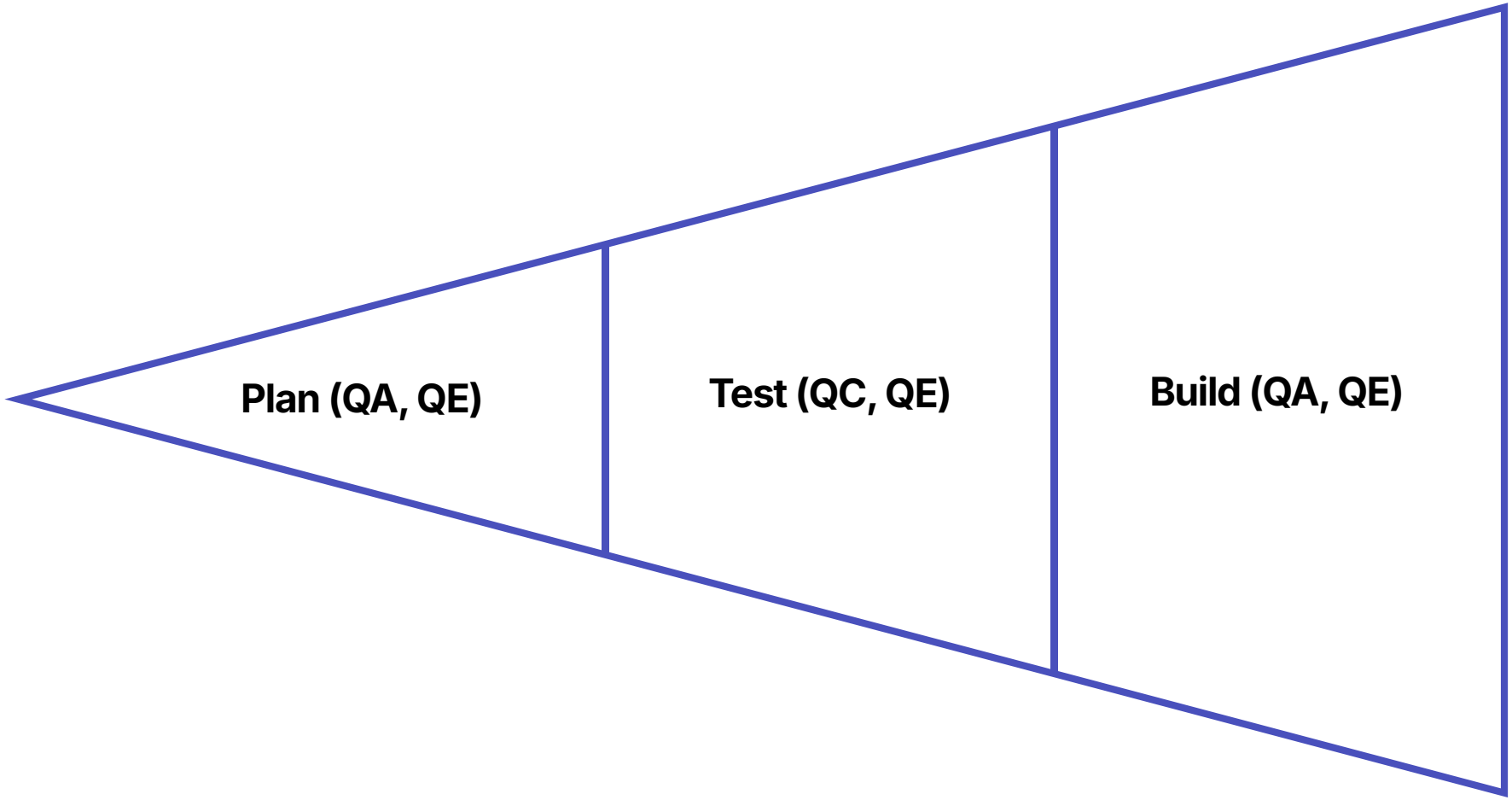
Software: Test execution, bug verification, release sign-off

QE — The Smart Kitchen

Temperature sensors, timers, and automated checks make it hard to produce a bad dish at scale.

Software: Automated pipelines, quality gates, monitoring

One Release — All Three Disciplines



QA, QC, and QE are not competitors. They act at different points and together create a quality safety net across the entire release lifecycle.

ACTIVITY · 10 MINUTES

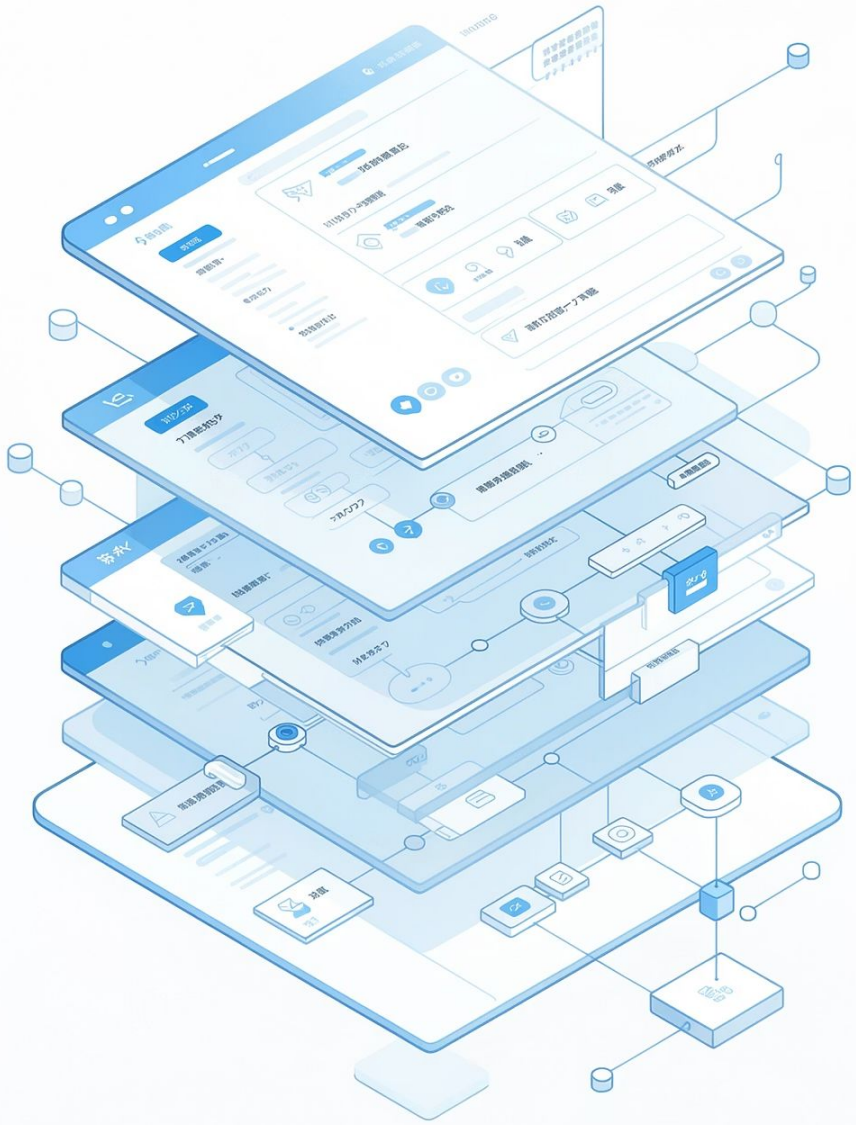
Classification Exercise

⚠ For each statement, identify whether it is QA, QC, or QE. Be ready to explain your answer.

#	Statement	Your Answer
1	The team writes automated regression tests that run on every pull request	QA / QC / QE?
2	A tester verifies that a bug fix works correctly before closing the ticket	QA / QC / QE?
3	The team adopts a definition of done that requires unit test coverage above 80%	QA / QC / QE?
4	A developer reviews a peer's code against the team's coding standards	QA / QC / QE?
5	A security scan is added as a mandatory gate in the CI pipeline	QA / QC / QE?
6	A tester executes smoke tests after every deployment to staging	QA / QC / QE?

Debrief: Classification Answers

#	Statement	Answer	Reasoning
1	Automated regression tests on every PR	**QE**	Engineering quality into the pipeline
2	Tester verifies a fix before closing	**QC**	Inspecting the product output
3	Definition of done with coverage threshold	**QA**	Defining the process standard
4	Code review against coding standards	**QA**	Ensuring process adherence
5	Security scan as a CI gate	**QE**	Automated quality gate in the pipeline
6	Smoke tests after deployment	**QC**	Detecting issues in the deployed product



SECTION 3 OF 5

Testing Fundamentals

What testing really means, the types you need to know, and how to design tests that actually find risk.

What Is Testing?

Common misconception

"Testing means finding bugs."

More accurate definition

Testing is the activity of **discovering risk** and **building confidence** that the system behaves correctly under defined conditions.

Find risk

Expose what might fail before users do

Build confidence

Show that known scenarios work correctly

Inform decisions

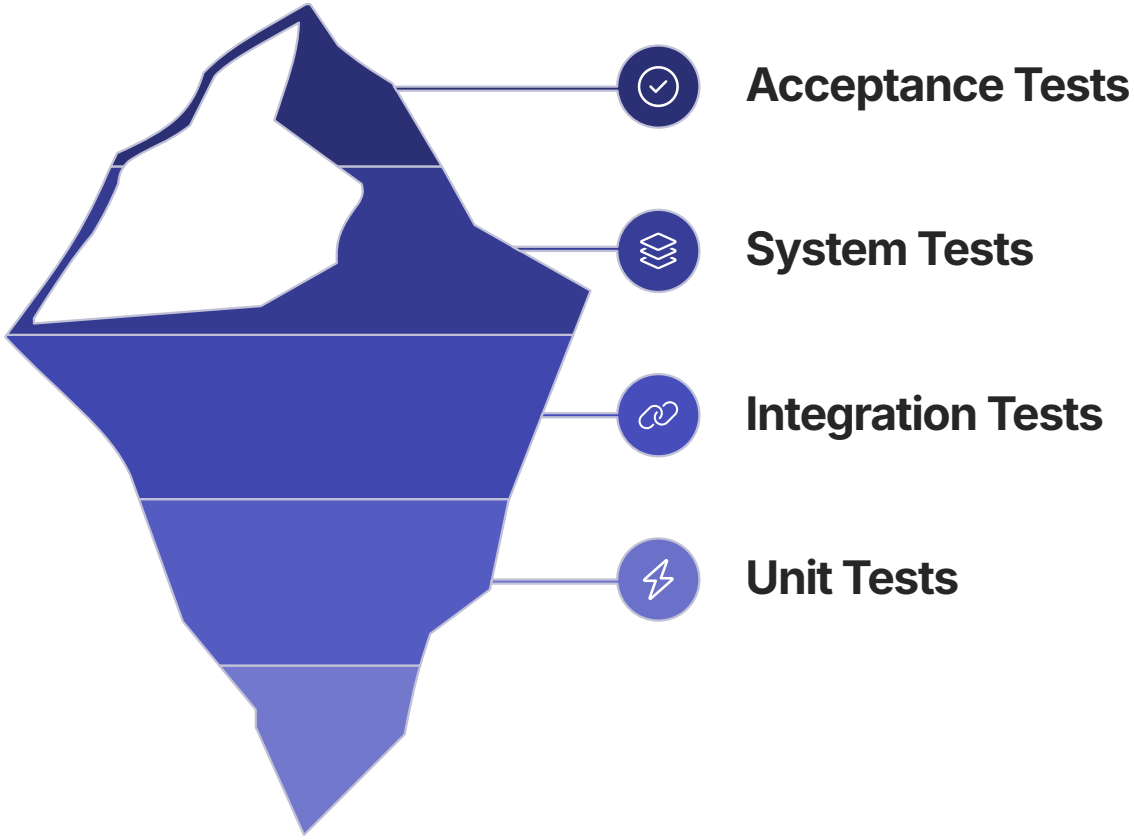
Help the team decide if it is safe to ship

Verification vs Validation

Verification	Validation
"Are we building it right?"	**"Are we building the right thing?"**
Checking that the system matches the specification	Checking that the system meets actual user needs
Example: Did the API return status 200 with the correct JSON schema?	Example: Does the refund flow make sense to the customer trying to use it?

Both are needed. A system can be technically correct but fail users completely.

Test Levels



Lower levels are faster, cheaper, and more isolated. Higher levels test real user journeys but run slower. Build confidence at every level.

Functional Testing

Testing *what* the system does

Login / Authentication

Can a valid user log in? Is an invalid user rejected?

Checkout / Payment

Does a transaction process correctly and produce a receipt?

Refund / Reversal

Is the correct amount returned to the correct account?

Search

Does the result set match filters and return within time?

Permissions

Can a read-only user not perform write actions?

Functional tests verify features. If a feature is broken, users notice immediately.

Non-Functional Testing

Testing *how well* the system works

Performance

Does it respond fast enough under real load?

Security

Is data protected from unauthorised access?

Usability

Can real users accomplish tasks without confusion?

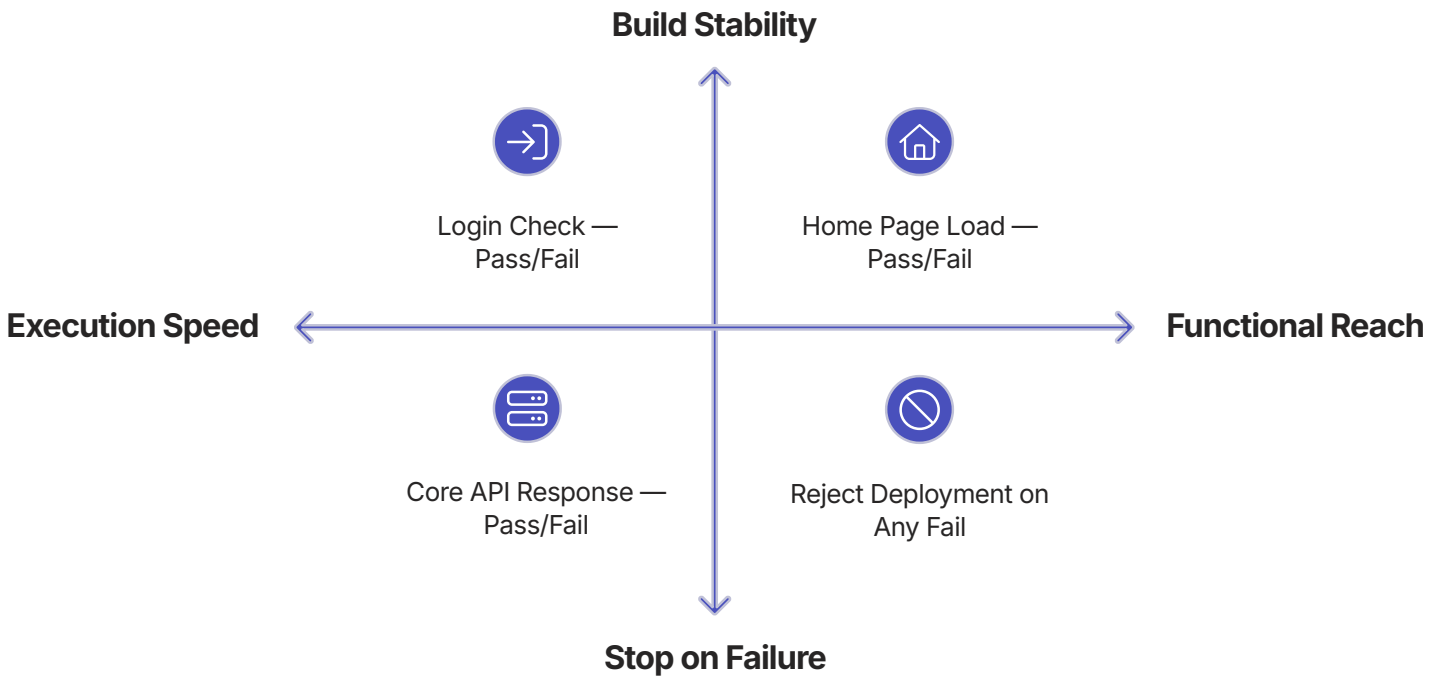
Reliability

Does it stay up and recover gracefully when it fails?

Compatibility

Does it work across browsers, devices, and OS versions?

Smoke Testing

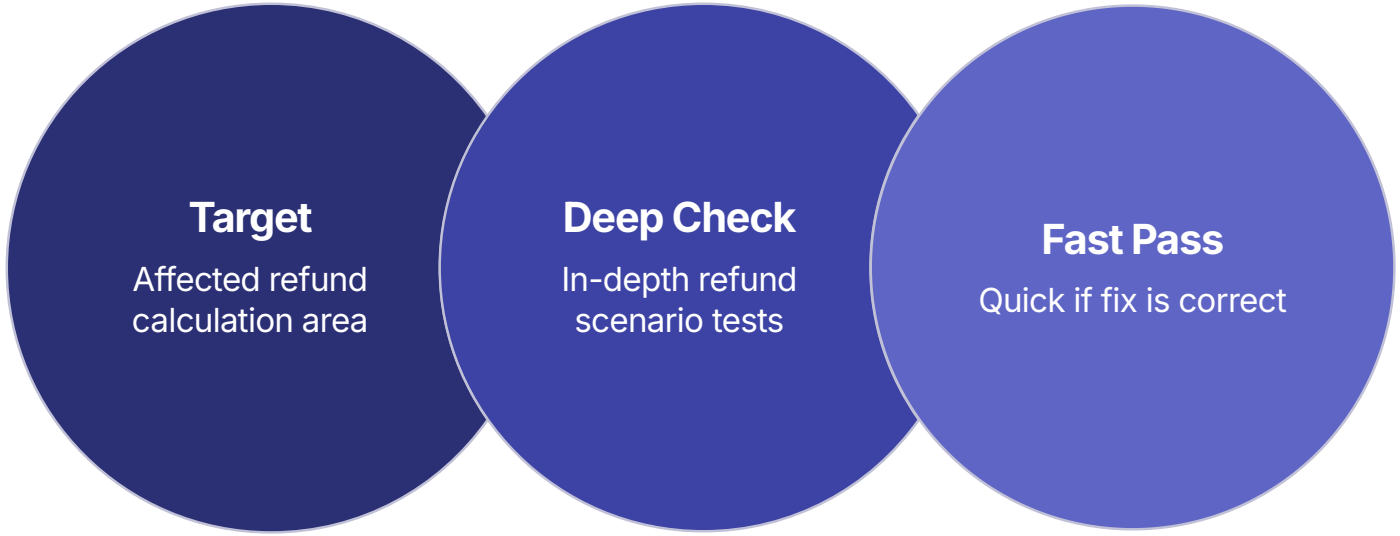


Broad. Shallow. Fast.

Smoke tests answer one question: **"Is this build stable enough to test further?"**

- Covers the widest surface area
- Runs in minutes after deploy
- Does not go deep into any one feature
- A fail here means stop — do not proceed

Sanity Testing



Narrow. Deep. Targeted.

Sanity tests answer: **"Did this specific fix or change work correctly?"**

- Runs after a small fix or patch
- Tests only the affected area deeply
- Saves time by not retesting everything
- Often manual or semi-automated

Regression Testing

Ensuring nothing is broken by a change

Regression tests answer: **"Does existing behaviour still work after this change?"**

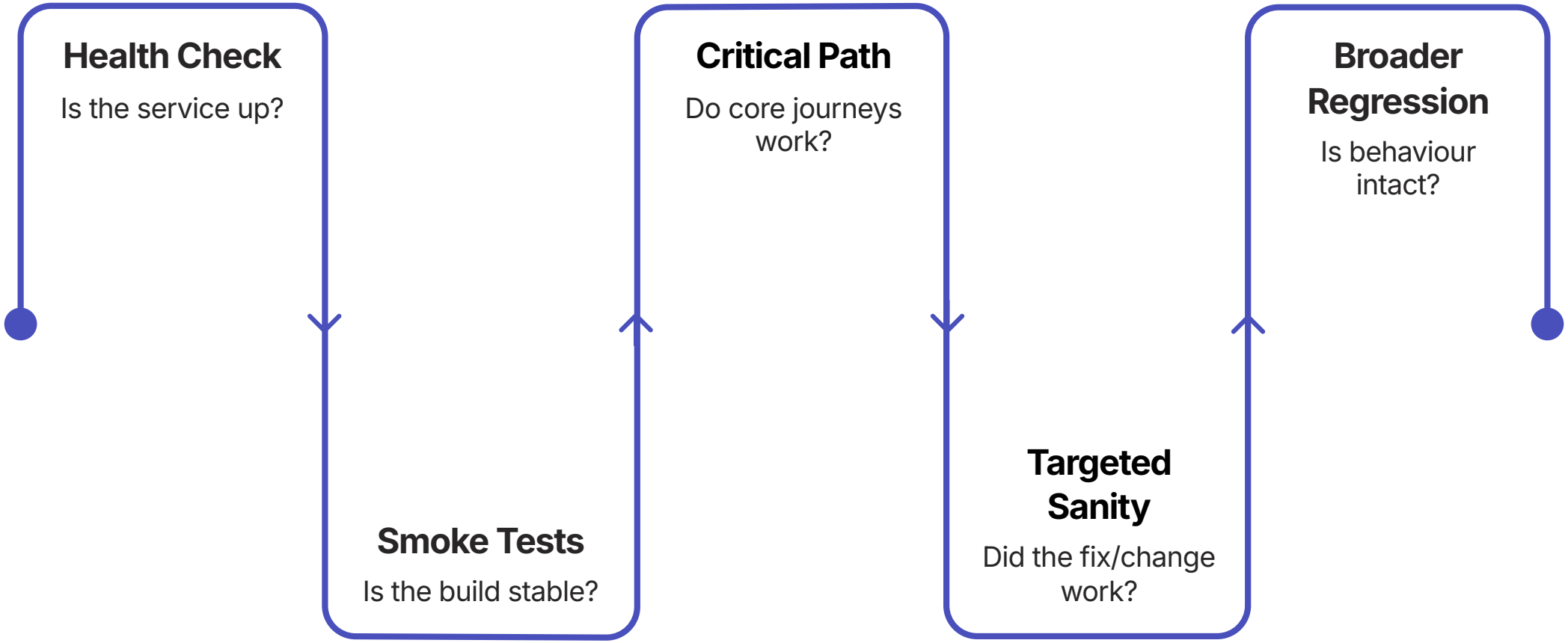
- Runs after any significant change
- Covers the existing feature set broadly
- Usually automated in sustain engineering
- The safety net for production changes

In sustain engineering, regression is your most important test type.

Smoke vs Sanity vs Regression

Dimension	Smoke	Sanity	Regression
Scope	Broad — whole system	Narrow — one area	Wide — existing features
Depth	Shallow	Deep	Moderate
Trigger	After every build/deploy	After a small fix or patch	After any significant change
Duration	Minutes	Minutes to 1 hour	Hours (automated)
Goal	Build is stable?	Fix is correct?	Nothing else broke?

After Deployment: What Do You Test First?



Follow this sequence every time. Do not skip steps. A failure at any stage is a signal to pause and investigate before proceeding.

Anatomy of a Good Test Case

01

ID & Title

Unique identifier and a clear, specific name — e.g. TC-042: Submit order with expired card

02

Objective

What is being validated and why it matters

03

Preconditions

What must be true before the test starts — user logged in, test data loaded

04

Steps & Test Data

Exact reproducible actions with the specific inputs used

05

Expected Result

The precise outcome that constitutes a pass — not "it works" but the exact behaviour

Positive, Negative, and Boundary Testing

Example: User login flow

Positive

Valid email + valid password

Expected: Login succeeds, user reaches dashboard

Negative

Valid email + wrong password

Expected: Login fails, clear error message, no access

Boundary

Password exactly at minimum length (8 chars) vs 7 chars

Expected: 8 chars accepted; 7 chars rejected with correct message

All three types are needed. Positive tests alone give false confidence.

Test Data Matters

Valid Data

Standard inputs that should work — confirms happy path

Invalid Data

Malformed, missing, or out-of-range inputs — confirms error handling

Edge / Boundary Data

Minimum, maximum, zero, empty — where systems often fail silently

Production-Like Data

Realistic volume and variety — reveals issues that toy data hides

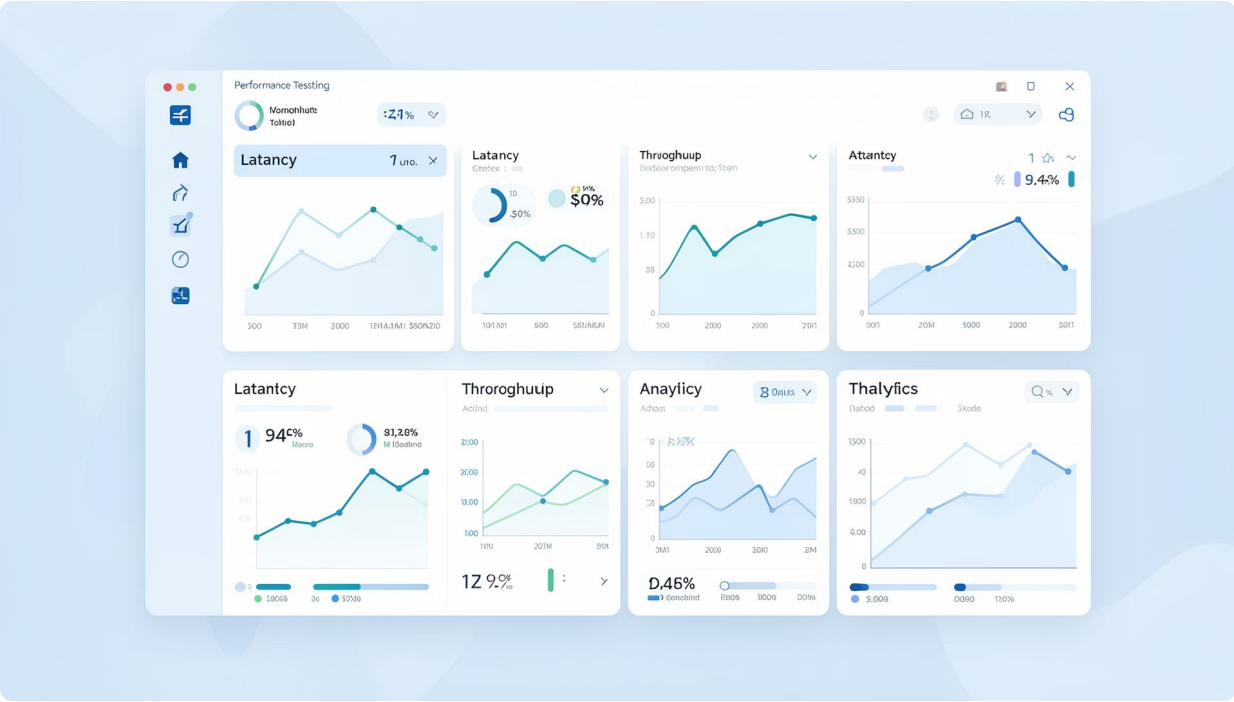
Privacy-Safe Data

Never use real customer PII in test environments — use anonymised sets

Test Case Design: Submit Transaction Flow

Type	Scenario	Input	Expected Result
Happy path	Valid order, sufficient balance	Item in stock, valid card	Order confirmed, charge applied once
Negative	Insufficient funds	Balance below order total	Order rejected, clear error message, no charge
Negative	Expired payment method	Card expiry in the past	Payment declined, user prompted to update card
Boundary	Maximum order quantity	Order at stock limit (e.g. 100 units)	Order accepted; 101 units rejected
Negative	Duplicate submission	Same request submitted twice rapidly	Only one charge processed, idempotency respected

Performance Testing: More Than Response Time



Why response time alone is not enough

- A page loads in 200ms for 1 user. What about 10,000 concurrent users?
- Averages hide outliers — the worst 1% of users may be experiencing 10-second delays
- Error rate can spike before response time degrades
- Memory leaks only appear after hours of sustained load

Performance testing validates behaviour under realistic conditions, not just ideal ones.

Key Performance Metrics

Latency — p50 / p95 / p99

Median, 95th percentile, and 99th percentile response times. p99 reveals the worst user experiences hidden by averages.

Throughput

Requests or transactions per second the system can handle before quality degrades.

Error Rate

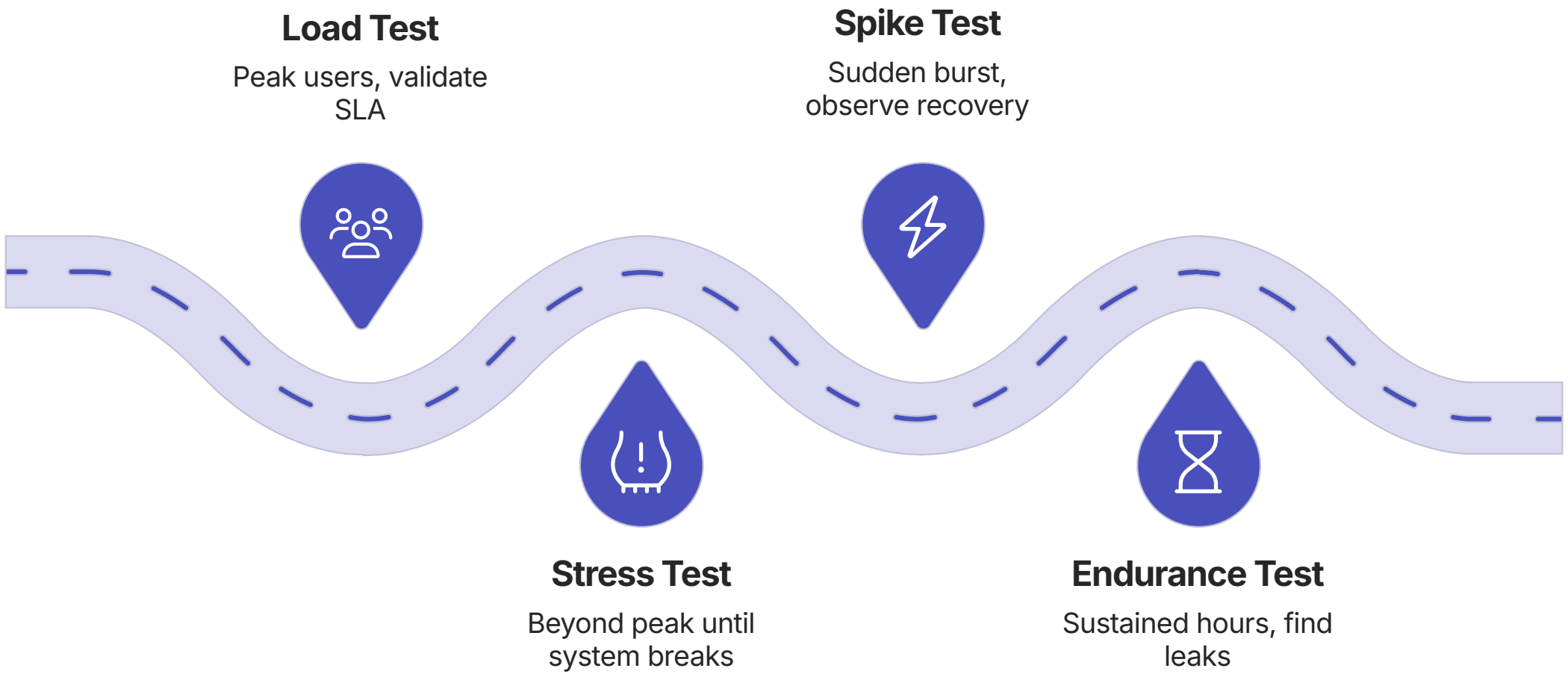
Percentage of requests that return an error. Often the first metric to degrade under stress.

Concurrency

Number of simultaneous active users the system can support without SLA breach.

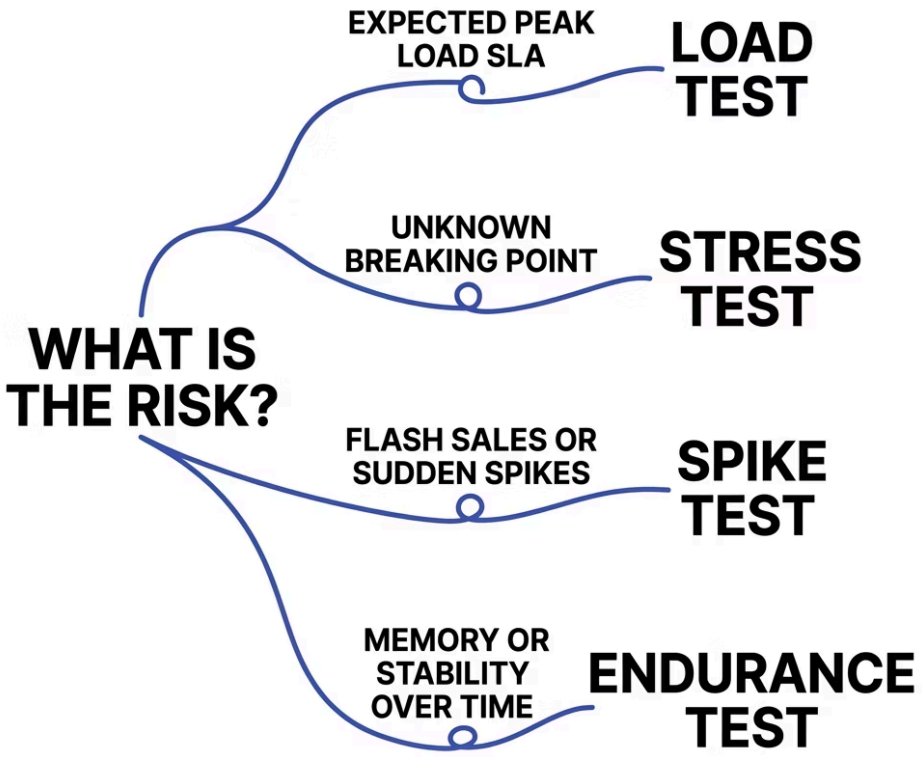
Always define your SLA thresholds before running a performance test. Without a target, a result means nothing.

Load vs Stress vs Spike vs Endurance



Start with load testing. Graduate to stress and spike only once the baseline is established. Run endurance tests periodically on sustain systems.

Which Performance Test for Which Risk?



Match the test to the risk

Not every feature needs all four test types. Ask: what real-world condition are we worried about?

- **New payment gateway** — load test first
- **Seasonal sale event** — spike test
- **Background job runs nightly** — endurance test
- **Unknown system limits** — stress test

SECTION 4 OF 5

Severity, Priority & Defect Management

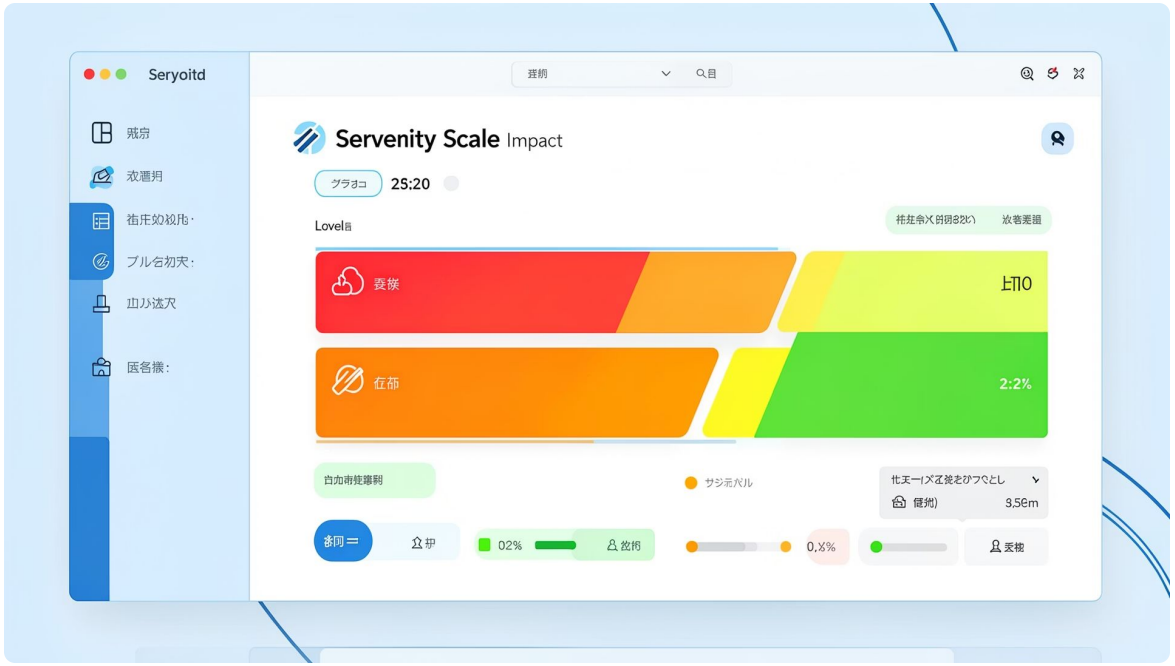
Not all bugs are equal. Learning to classify and manage defects correctly is a core sustain engineering skill.

Severity: How Bad Is the Damage?

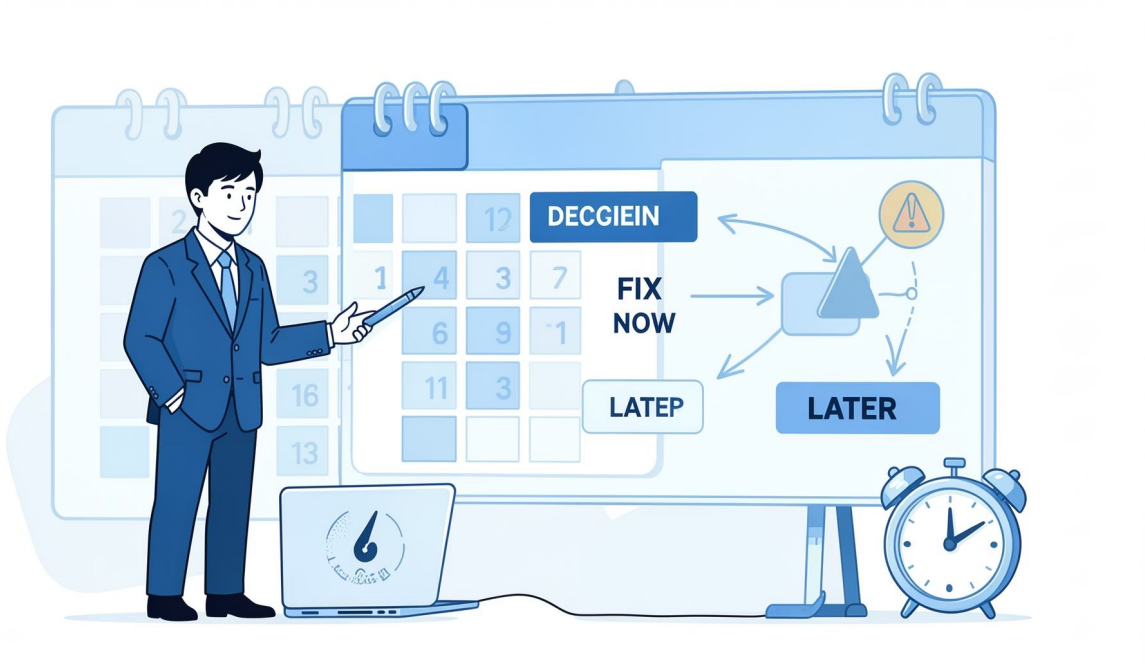
Severity = impact on the system, user, or business

Severity is a **technical assessment**. It does not depend on when you need to fix it.

Level	Meaning
Critical	System down or data loss
Major	Core feature broken
Minor	Feature impaired, workaround exists
Low	Cosmetic or negligible impact



Priority: How Urgently Must We Fix It?

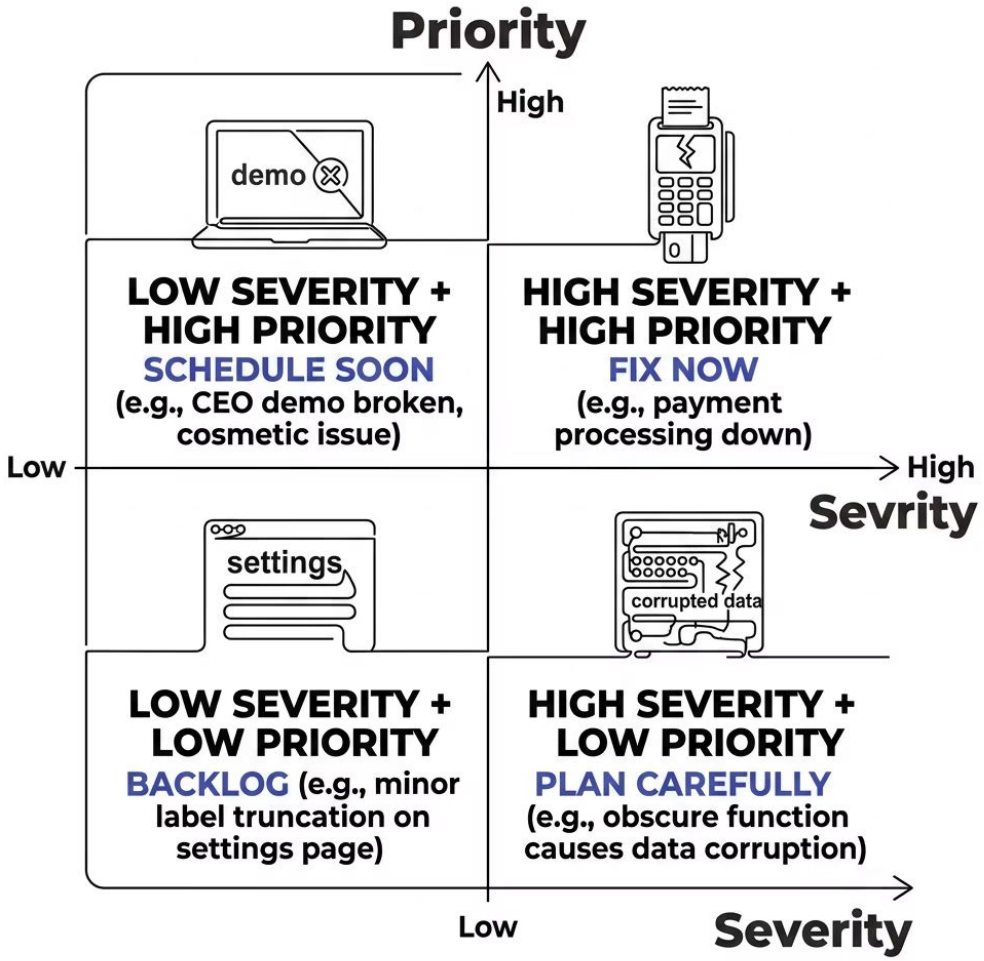


Priority = business urgency of the fix

Priority is a **business decision**. It can be influenced by releases, SLAs, and customer impact.

Level	Meaning
P1	Fix immediately — production blocked
P2	Fix in this sprint
P3	Fix in next sprint
P4	Fix when time permits

Severity × Priority Matrix

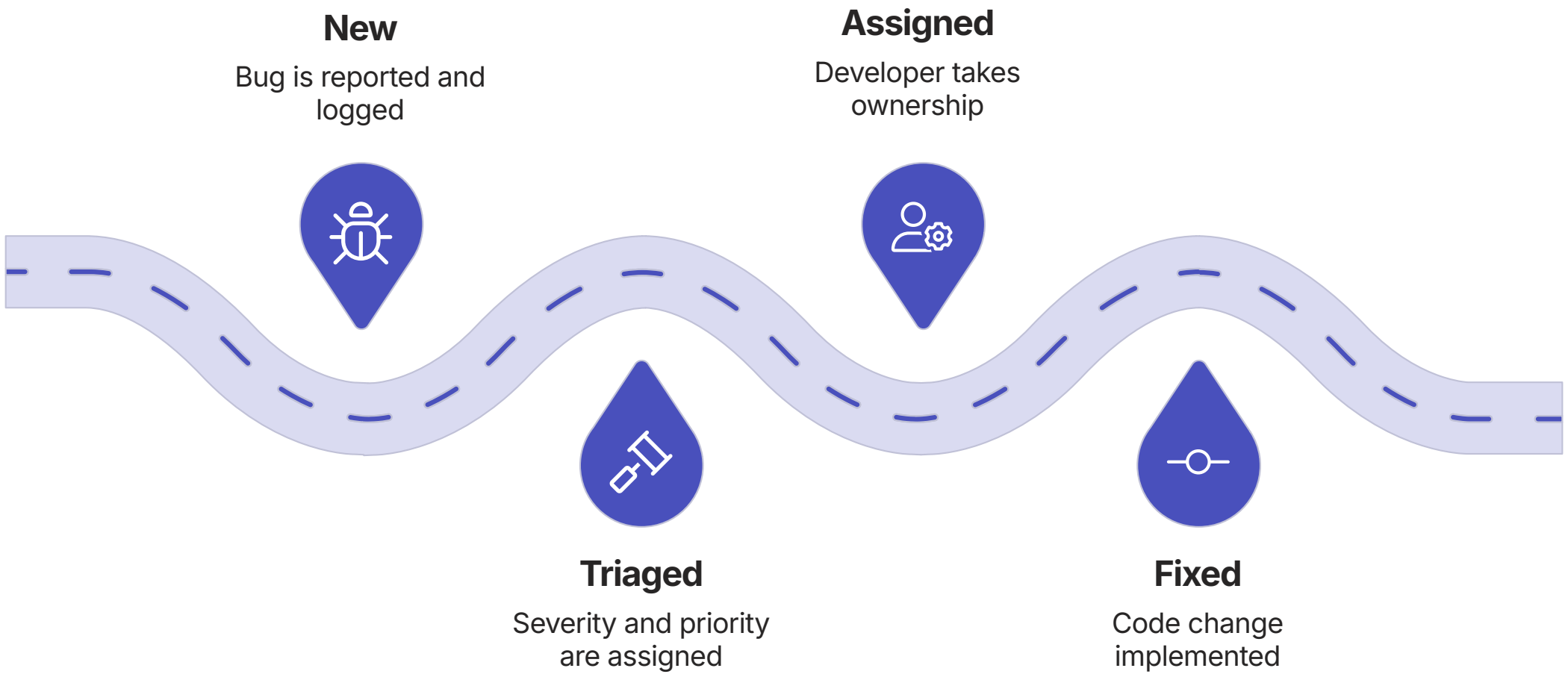


Why this matrix matters

- High severity + high priority: drop everything and fix
- High severity + low priority: mitigate now, fix planned
- Low severity + high priority: schedule before next release
- Low severity + low priority: log it, revisit in backlog

Severity and priority can differ. A cosmetic bug on the CEO's demo screen may be low severity but high priority.

The Defect Lifecycle



Every bug should move through this lifecycle explicitly. Bugs that skip steps cause rework, confusion, and missed regressions in production.

Anatomy of a Good Bug Report

01

Title

Specific and searchable — e.g. "Duplicate charge triggered on retry after timeout"

02

Environment

Production / Staging, browser version, OS, region, user role

03

Steps to Reproduce

Exact, numbered steps that anyone can follow to trigger the bug

04

Expected vs Actual

What should happen vs what actually happened — be precise

05

Evidence & Impact

Screenshots, logs, trace IDs. How many users affected? What is the business impact?

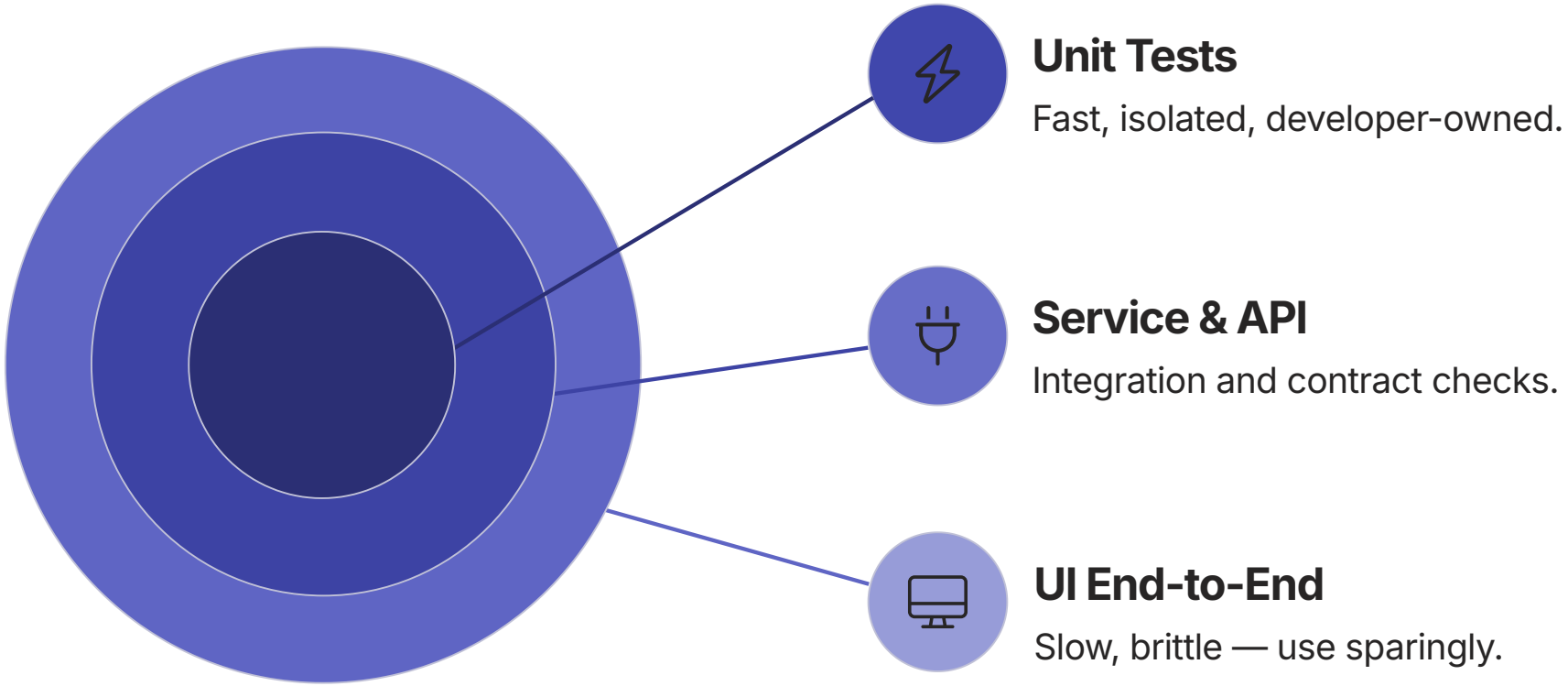
Production Bug Report Example

Field	Content
Title	Duplicate charge triggered when payment API times out and user retries
Environment	Production — Payment Service v2.4.1 — EU-West region
Steps	1. User initiates checkout. 2. Payment API times out at 30s. 3. User clicks Retry. 4. Two charges appear on the user's statement.
Expected	Retry should detect the pending transaction and not create a second charge
Actual	A second charge is created. The original timed-out transaction also settles. Both charges are collected.
Evidence	Trace ID: TXN-8821X, attached logs, 47 customer complaints in 2 hours
Impact	Severity: Critical | Priority: P1 — financial harm to customers, refund cost to business

SECTION 5 OF 5

Automation, Quality Gates & Metrics

Building quality into the pipeline — so it is not an afterthought but a continuous safety net.



Unit Tests

Fast, isolated, developer-owned.

Service & API

Integration and contract checks.

UI End-to-End

Slow, brittle — use sparingly.

Invest most in the bottom layers. A top-heavy pyramid is slow, fragile, and expensive to maintain.

Manual vs Automation: When to Use Each

Scenario	Manual	Automate
Exploratory investigation	Yes — human judgement required	No
Regression after every deploy	No — too slow and error-prone	Yes — consistent and fast
New, unstable feature	Yes — spec may change frequently	Not yet
Stable, high-frequency flow	No — waste of manual effort	Yes — high return on investment
High-risk critical path	Supplement — spot checks	Yes — must not be manual-only
Visual / UX validation	Yes — humans judge aesthetics	Partial — screenshot comparison only

CI/CD Quality Gates

Every gate is a quality checkpoint. A failure at any gate stops the pipeline. This is intentional — catching issues here is always cheaper than in production.

Quality Is Built In — Not Bolted On

Risk-Based Testing

Test what matters most. Focus on high-severity, high-likelihood failure points first.

Shift-Left

Involve quality thinking early — at design and requirements, not just at deploy.

Key Metrics to Watch

Defect escape rate, test coverage, p99 latency, error rate, MTTR after incidents.

Sustain Engineer Checklist

Before every change: understand impact, write test cases, run regression, observe post-deploy, document findings.

Your takeaway from Module 13

Change safely. Prove it. Observe it. Learn from it.

Up Next

Module 14 — Infrastructure Fundamentals

Where your applications live, how they scale, and how infrastructure decisions shape quality.